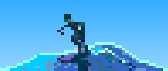


4 实现多线程的两种方法





7.4.1 Thread 类—实现多线程的基础类



创建线程需要通过调用 **Thread** 类的构造方法来实现。

Thread 类的常用构造方法：

(1) `public Thread()`

(2) `public Thread(Runnable target)`

(3) `public Thread(String name)`

(4) `public Thread(Runnable target , String name)`

- **name**: 指定了线程的名称
- **target**: 实现 `Runnable` 接口的对象，也就是提供线程体的对象。

`java.lang.Runnable` 接口中定义了 `run()` 方法，实现该接口的类需要通过实现 `run()` 方法来提供线程体，当线程运行时，`run()` 方法将被系统自动调用。



7.4.2 Runnable 接口—提供了线程的具体实现

- 线程创建中，线程体的构造是关键。任何实现 **Runnable** 接口的对象都可以作为 **Thread** 类构造方法中的 **target** 参数，而 **Thread** 类本身也实现了 **Runnable** 接口。

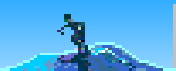


- 因此可以有两种方式提供 **run()** 方法的实现 -- 也就是创建线程体

(1) 实现 **Runnable** 接口 ; (2) 继承 **Thread** 类



7.4.3 通过实现 **Runnable** 接口实现多线程



在 java.lang 包中 Runnable 接口的定义为：

```
public interface Runnable {  
    public void run();  
}
```

使用这种方式创建线程的步骤如下：

步骤一

首先定义一个类，让其实现
Runnable 接口，在该类中提供
run() 方法的实现

步骤二

然后把该类的一个实例作为参数传
递给 **Thread** 类的构造方法，来创
建 **Thread** 对象



例： 通过实现 **Runnable** 接口实现多线程

```
class Hello implements Runnable
{   int i;
    public void run() {
        while(true) {
            System.out.println("Hello"+i++);
            if(i==5) break;
        }
    }
}
```

```
public class ThreadTest
{   public static void main(String args[])
    {
        Thread t1=new Thread(new Hello());
        t1.start();
    }
}
```

Hello 类实现了 **Runnable** 接口，
并重写其中的 **run()** 方法。

```
Hello0
Hello1
Hello2
Hello3
Hello4
```

新建的线程 **t1** 不会自动运行，必须
调用 **start()** 方法，才让其可被调度
运行。

t1 运行后，执行的就是 **Hello** 对象
中的 **run()** 方法。



7.4.4 通过继承 Thread 类实现多线程

在 java.lang 包中, Thread 类的声明如下:

```
public class Thread extends Object implements Runnable
```

Thread 类本身已经实现了 **Runnable** 接口。

通过继承 **Thread** 类创建线程的步骤如下:

步骤一

首先从 **Thread** 类派生子类, 并通过重写其中的 **run()** 方法来定义线程体。

步骤二

然后通过创建该子类的对象来创建线程。



例：通过继承 Thread 类实现多线程

```
class Hello extends Thread
{   int i;
    public void run()   {
        while(true) {
            System.out.println("Hello"+i+
+);
            if(i==5) break;
        }
    }

    public class ThreadTest2
    {   public static void main(String args[])
        {
            Hello t1=new Hello();
            t1.start();
        }
    }
```

类 Hello 继承了 Thread 类，并对 run() 方法进行了重写。

```
Hello0
Hello1
Hello2
Hello3
Hello4
```

在主类的 main() 方法中，创建 Hello 的实例 t1，并调用其 start() 方法，让其可被调度运行。线程对象 t1 运行后，执行的就是 run() 方法。



实现多线程的两种方法的比较——采用继承 **Thread** 类的方法

势

编写简单，在 `run()` 方法中如果需要调用当前线程的其他方法，只需使用 `this` 即可，无需使用 `Thread.currentThread()` 方法。

优

劣势

因为这种线程类已经继承了 **Thread** 类，所以不能再继承其它类。



实现多线程的两种方法的比较——实现 **Runnable** 接口的方法

优势

线程类只是实现了 **Runnable** 接口，因此还可以继承其他类；在这种情况下，可以使多个线程共享一个 **target** 对象，所以非常适合多个线程用来处理同一份资源的情况，模型清晰，更加符合面向对象的设计思想。

劣势

编程略有些复杂，如果要访问当前线程必须使用 **Thread.currentThread()** 方法。如：想获得当前活动线程的名称，用语句：

```
Thread.currentThread().getName()
```

提倡采用通过实现 **Runnable** 接口的方式来实现多线程。

具体应用中，可以根据实际情况确定采用哪种方法。





本节内容结束，谢谢观看！

